

# ADR-002: Elección de Arquitectura – Hexagonal vs Clean Architecture

Estado

Aceptado

Fecha

19/06/2026

Decisión

Adoptar **Arquitectura Hexagonal (Ports & Adapters)** como estilo arquitectónico principal para el desarrollo de la solución.

## 1. Contexto

El proyecto requiere construir una solución desacoplada, mantenible y escalable, capaz de integrarse con múltiples sistemas externos, bases de datos, servicios APIs REST y servicios cloud.

Se busca una arquitectura que permita:

- Separar claramente la lógica de negocio de los detalles tecnológicos.
- Facilitar las pruebas unitarias e integración.
- Permitir la sustitución de componentes de infraestructura con el menor impacto posible.
- Reducir el acoplamiento entre las reglas de negocio y los frameworks utilizados.
- Mantener una estructura comprensible para el equipo de desarrollo.

Para ello se evaluaron dos alternativas arquitectónicas: Arquitectura Hexagonal y Clean Architecture.

## 2. Decisión

Se adopta **Arquitectura Hexagonal (Ports & Adapters)** como arquitectura base para el proyecto.

La solución organizará sus componentes mediante puertos y adaptadores, manteniendo el dominio aislado de dependencias externas y permitiendo la evolución independiente de las capas tecnológicas.

### 3. Alternativas Consideradas

#### A. Arquitectura Hexagonal (Seleccionada)

##### Pros

- Separación clara entre dominio, aplicación e infraestructura.
- Facilita la sustitución de tecnologías sin afectar las reglas de negocio.
- Alta testabilidad mediante el uso de puertos e interfaces.
- Reduce el acoplamiento con frameworks y proveedores externos.
- Estructura simple y fácil de comprender para el equipo.
- Se adapta naturalmente al desarrollo de microservicios y APIs con Spring Boot.
- Permite aplicar principios SOLID e inversión de dependencias.

##### Contras

- Requiere disciplina en la definición de puertos y adaptadores.
- Puede generar un mayor número de interfaces respecto a arquitecturas tradicionales.

#### B. Clean Architecture

##### Pros

- Alto nivel de desacoplamiento entre capas.
- Independencia tecnológica.
- Excelente soporte para pruebas unitarias.
- Fuerte orientación al dominio.

##### Contras

- Mayor complejidad estructural.
- Incremento en el número de capas y componentes.
- Curva de aprendizaje superior para el equipo.
- Riesgo de sobreingeniería para proyectos de tamaño medio.
- Mayor esfuerzo de mantenimiento debido a la cantidad de abstracciones.

### 4. Consecuencias

##### Positivas

- Aislamiento efectivo de la lógica de negocio.
- Facilidad para incorporar nuevos adaptadores de entrada o salida.
- Menor impacto ante cambios tecnológicos futuros.
- Mejor mantenibilidad del código.

- Mayor facilidad para pruebas automatizadas.
- Estructura consistente para todos los módulos del sistema.

#### Negativas

- Necesidad de capacitación inicial del equipo en el patrón Ports & Adapters.
- Incremento moderado en la cantidad de interfaces y clases de infraestructura.
- Mayor esfuerzo inicial de diseño respecto a una arquitectura tradicional en capas.

#### 5. Estado Final

Se aprueba el uso de **Arquitectura Hexagonal** como estándar arquitectónico para el proyecto.

Los nuevos módulos deberán organizarse siguiendo el modelo de Dominio, Aplicación y Adaptadores, garantizando que las reglas de negocio permanezcan independientes de frameworks, bases de datos, servicios externos y tecnologías de infraestructura.